

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.5

Aim : Support Vector Machines (SVM)

- Implement the SVM algorithm for binary classification.
- Train an SVM model using a given dataset and optimize its parameters.
- Evaluate the performance of the SVM model on test data and analyze the results.

Code:

```
pr05.py - C:\Users\gazala\OneDrive\Desktop\pr05.py [3.13.2]
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the Iris dataset
df = pd.read_csv('Iris.csv')

# Select only two classes for binary classification
df = df[df['Species'].isin(['Iris-setosa', 'Iris-versicolor'])]

# Select two features for classification and visualization
X = df[['PetalLengthCm', 'PetalWidthCm']]
y = df['Species']

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Standardize the feature values
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.5

```
pr05.py - C:\Users\gazala\OneDrive\Desktop\pr05.py [1.13.2]
File Edit Format Run Options Window Help

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define different SVM parameters for optimization
parameters = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto'],
    'kernel': ['linear', 'rbf']
}

# Find the best combination of parameters
grid_search = GridSearchCV(
    SVC(),
    parameters,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)

# Display the best parameters
print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(grid_search.best_score_)

# Train the SVM model using the best parameters
model = grid_search.best_estimator_

# Predict the test data
y_pred = model.predict(X_test)

# Convert test labels into numeric values
test_labels = np.where(y_test == classes[0], 0, 1)

# Plot the SVM decision boundary
plt.figure(figsize=(9, 6))

plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(
    X_train[:, 0],
    X_train[:, 1],
    c=train_labels,
    edgecolors='black',
    label='Training Data'
)

plt.scatter(
    X_test[:, 0],
    X_test[:, 1],
    c=test_labels,
    marker='x',
    s=80,
    label='Test Data'
)

plt.xlabel('Standardized Petal Length')
plt.ylabel('Standardized Petal Width')
plt.title('SVM Decision Boundary')
plt.legend()

plt.show()
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.5

Output:

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
----- RESTART: C:/Users/gazala/OneDrive/Desktop/pno5.py -----
Best Parameters:
{'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}

Best Cross-Validation Accuracy:
1.0

Test Accuracy: 1.0
Test Accuracy Percentage: 100.0 %

Confusion Matrix:
[[10  0]
 [ 0 10]]

Classification Report:
              precision    recall  f1-score   support

 Iris-setosa      1.00      1.00      1.00        10
 Iris-versicolor  1.00      1.00      1.00        10

   accuracy
   macro avg      1.00      1.00      1.00        20
  weighted avg      1.00      1.00      1.00        20
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.5

